

Sztuczna Inteligencja i Inżynieria Wiedzy - Lista nr 3

Planowanie z wykorzystaniem języka PDDL

Mikołaj Kubś

14 maja 2025

Spis treści

1	Wprowadzenie	3
2	Zadanie 1: Transport paczek z rozszerzeniami	3
2.1	Opis problemu i założenia modelu	3
2.2	Definicja domeny (<code>domain-task1.pddl</code>)	3
2.2.1	Typy	5
2.2.2	Predykaty	5
2.2.3	Funkcje (fluenty numeryczne)	5
2.2.4	Akcje (Przykłady)	6
2.3	Definicja problemu (<code>problem-task1.pddl</code>)	6
2.3.1	Stan początkowy	8
2.3.2	Cel i metryka	8
2.4	Wygenerowany plan	8
2.5	Analiza planu i eksperymenty	9
3	Zadanie 2: Sprzątający robot	9
3.1	Opis problemu	9
3.2	Definicja domeny (<code>domain-task2.pddl</code>)	9
3.3	Definicja problemu (<code>problem-task2.pddl</code>)	10
3.4	Wygenerowany plan	11
3.5	Analiza planu	12
4	Zadanie 3: Robot z dwoma ramionami przenoszący piłki	12
4.1	Opis problemu	12
4.2	Analiza dostarczonych plików PDDL	12
4.3	Wygenerowany plan	12
4.4	Analiza planu	13
5	Wnioski	13

1 Wprowadzenie

Celem niniejszego raportu jest przedstawienie rozwiązań trzech zadań z zakresu planowania z wykorzystaniem języka PDDL (Planning Domain Definition Language). Lista zadań miała na celu zapoznanie z podstawami języka PDDL, jego rozszerzeniami oraz praktycznym zastosowaniem w modelowaniu i rozwiązywaniu problemów planowania symbolicznego. W ramach zadań zaimplementowano modele dla problemu transportu paczek z rozszerzeniami, sprząającego robota oraz przeanalizowano gotowy model robota przenoszącego piłki. Do testowania i generowania planów wykorzystano edytor online <https://editor.planning.domains>.

2 Zadanie 1: Transport paczek z rozszerzeniami

2.1 Opis problemu i założenia modelu

Zadanie polegało na zaprojektowaniu, uruchomieniu i analizie planu transportu paczek. Celem było opracowanie modelu logicznego oraz wdrożenie go w formie plików PDDL, uwzględniając następujące rozszerzenia języka PDDL:

- `:strips` - podstawowy model operacji STRIPS,
- `:typing` - typowanie obiektów (np. paczka, lokacja, pojazd),
- `:negative-preconditions` - warunki negatywne,
- `:conditional-effects` - efekty warunkowe (np. dodatkowy koszt dla paczek delikatnych),
- `:numeric-fluents` i `:action-costs` - koszty działań,
- `:durative-actions` - czas trwania akcji,
- Złożona topologia transportu (drogowego, lotniczego oraz wodnego).
- Model uwzględniał również możliwość działania wielu pojazdów (aspekt `:multi-agent` realizowany poprzez definicję wielu obiektów typu pojazd i możliwość ich równoległego (w sensie czasowym) działania w planie).

Model został rozwijany iteracyjnie, dodając kolejne funkcjonalności i rozszerzenia, co pozwoliło na stopniowe testowanie i weryfikację poprawności.

2.2 Definicja domeny (domain-task1.pddl)

```
1 (define (domain package-transport-step6)
2   (:requirements :strips :typing :negative-preconditions :action-costs
3     :durative-actions :conditional-effects)
4
5   (:types
6     package
7     location
8       city airport port - location
9     vehicle
10      truck plane ship - vehicle
11   )
12
13   (:predicates
14     (at-pkg ?p - package ?l - location)
15     (at-vehicle ?v - vehicle ?l - location)
16     (in-pkg ?p - package ?v - vehicle)
17     (road-connection ?from - location ?to - location)
```

```

18 (air-connection ?from - airport ?to - airport)
19 (sea-connection ?from - port ?to - port)
20 (fragile ?p - package)
21 )
22
23 (:functions
24 (total-cost) - number
25 (travel-distance ?l1 - location ?l2 - location) - number
26 (speed ?v - vehicle) - number
27 (load-unload-fixed-time) - number
28 )
29
30 (:durative-action load-package
31 :parameters (?pkg - package ?veh - vehicle ?loc - location)
32 :duration (= ?duration (load-unload-fixed-time))
33 :condition (and
34 (at start (at-pkg ?pkg ?loc))
35 (at start (at-vehicle ?veh ?loc))
36 )
37 :effect (and
38 (at end (in-pkg ?pkg ?veh))
39 (at end (not (at-pkg ?pkg ?loc)))
40 ; Conditional costs
41 (when (and (at start (fragile ?pkg))) ; If fragile
42 (at end (increase (total-cost) 15))) ; Total load cost for
43 (fragile = 5 (base) + 10 (extra)
44 (when (and (at start (not (fragile ?pkg)))) ; If NOT fragile
45 (requires :negative-preconditions also for conditions in
46 'when')
47 (at end (increase (total-cost) 5)))
48 )
49 )
50 )
51
52 (:durative-action unload-package
53 :parameters (?pkg - package ?veh - vehicle ?loc - location)
54 :duration (= ?duration (load-unload-fixed-time))
55 :condition (and
56 (at start (in-pkg ?pkg ?veh))
57 (at start (at-vehicle ?veh ?loc))
58 )
59 :effect (and
60 (at end (at-pkg ?pkg ?loc))
61 (at end (not (in-pkg ?pkg ?veh)))
62 (at end (increase (total-cost) 3))
63 )
64 )
65
66 (:durative-action drive-truck
67 :parameters (?tr - truck ?from - location ?to - location)
68 :duration (= ?duration (/ (travel-distance ?from ?to) (speed ?tr)))
69 :condition (and
70 (at start (at-vehicle ?tr ?from))
71 (over all (road-connection ?from ?to))
72 )
73 :effect (and
74 (at start (not (at-vehicle ?tr ?from)))
75 (at end (at-vehicle ?tr ?to))
76 (at end (increase (total-cost) 10))
77 )
78 )
79
80 (:durative-action fly-plane
81 :parameters (?pl - plane ?from - airport ?to - airport)

```

```

78 :duration (= ?duration (/ (travel-distance ?from ?to) (speed ?pl)))
79 :condition (and
80     (at start (at-vehicle ?pl ?from))
81     (over all (air-connection ?from ?to))
82 )
83 :effect (and
84     (at start (not (at-vehicle ?pl ?from)))
85     (at end (at-vehicle ?pl ?to))
86     (at end (increase (total-cost) 100))
87 )
88 )
89
90 (:durative-action sail-ship
91 :parameters (?sh - ship ?from - port ?to - port)
92 :duration (= ?duration (/ (travel-distance ?from ?to) (speed ?sh)))
93 :condition (and
94     (at start (at-vehicle ?sh ?from))
95     (over all (sea-connection ?from ?to))
96 )
97 :effect (and
98     (at start (not (at-vehicle ?sh ?from)))
99     (at end (at-vehicle ?sh ?to))
100     (at end (increase (total-cost) 50))
101 )
102 )
103 )

```

Listing 1: Domena zadania 1

2.2.1 Typy

Zdefiniowano hierarchię typów dla lokacji i pojazdów:

```

1 (:types
2   package
3   location
4     city airport port - location
5   vehicle
6     truck plane ship - vehicle
7 )

```

Listing 2: Typy w domenie Zadania 1

2.2.2 Predykaty

Kluczowe predykaty obejmowały:

```

1 (at-pkg ?p - package ?l - location)
2 (at-vehicle ?v - vehicle ?l - location)
3 (in-pkg ?p - package ?v - vehicle)
4 (road-connection ?from - location ?to - location)
5 (air-connection ?from - airport ?to - airport)
6 (sea-connection ?from - port ?to - port)
7 (fragile ?p - package)

```

Listing 3: Predykaty w domenie Zadania 1

2.2.3 Funkcje (fluenty numeryczne)

Zdefiniowano następujące fluenty numeryczne:

```

1 (:functions
2   (total-cost) - number
3   (travel-distance ?l1 - location ?l2 - location) - number
4   (speed ?v - vehicle) - number
5   (load-unload-fixed-time) - number
6 )

```

Listing 4: Funkcje w domenie Zadania 1

2.2.4 Akcje (Przykłady)

Przykładowa akcja load-package z efektem warunkowym oraz akcja drive-truck z czasem trwania:

```

1 (:durative-action load-package
2   :parameters (?pkg - package ?veh - vehicle ?loc - location)
3   :duration (= ?duration (load-unload-fixed-time))
4   :condition (and
5     (at start (at-pkg ?pkg ?loc))
6     (at start (at-vehicle ?veh ?loc))
7   )
8   :effect (and
9     (at end (in-pkg ?pkg ?veh))
10    (at end (not (at-pkg ?pkg ?loc)))
11    (when (at start (fragile ?pkg))
12      (at end (increase (total-cost) 10)))
13    (at end (increase (total-cost) 5))
14  )
15 )

```

Listing 5: Akcja load-package z efektem warunkowym

```

1 (:durative-action drive-truck
2   :parameters (?tr - truck ?from - location ?to - location)
3   :duration (= ?duration (/ (travel-distance ?from ?to) (speed ?tr)))
4   :condition (and
5     (at start (at-vehicle ?tr ?from))
6     (over all (road-connection ?from ?to))
7   )
8   :effect (and
9     (at start (not (at-vehicle ?tr ?from)))
10    (at end (at-vehicle ?tr ?to))
11    (at end (increase (total-cost) 10))
12  )
13 )

```

Listing 6: Akcja drive-truck z czasem trwania

2.3 Definicja problemu (problem-task1.pddl)

```

1 (define (problem multi-modal-transport-conditional)
2   (:domain package-transport-step6)
3
4   (:objects
5     p1 - package
6     p2 - package
7
8     london paris newyork - city
9     heathrow jfk charlesdegaulle - airport
10    dover calais newyork_port - port

```

```

11
12 my_truck - truck
13 my_plane - plane
14 my_ship - ship
15 another_truck - truck
16 )
17
18 (:init
19   (= (total-cost) 0)
20   (= (load-unload-fixed-time) 1)
21
22   (= (speed my_truck) 60)
23   (= (speed another_truck) 60)
24   (= (speed my_plane) 500)
25   (= (speed my_ship) 30)
26
27   (= (travel-distance london heathrow) 30) (= (travel-distance heathrow
28     london) 30)
29   (= (travel-distance london dover) 80) (= (travel-distance dover london) 80)
30   (= (travel-distance paris charlesdegaulle) 20) (= (travel-distance
31     charlesdegaulle paris) 20)
32   (= (travel-distance paris calais) 150) (= (travel-distance calais paris)
33     150)
34   (= (travel-distance newyork jfk) 25) (= (travel-distance jfk newyork) 25)
35   (= (travel-distance newyork newyork_port) 10) (= (travel-distance
36     newyork_port newyork) 10)
37   (= (travel-distance jfk newyork_port) 30) (= (travel-distance newyork_port
38     jfk) 30)
39   (= (travel-distance london paris) 350) (= (travel-distance paris london)
40     350)
41   (= (travel-distance heathrow jfk) 3000) (= (travel-distance jfk heathrow)
42     3000)
43   (= (travel-distance heathrow charlesdegaulle) 300) (= (travel-distance
44     charlesdegaulle heathrow) 300)
45   (= (travel-distance dover calais) 50) (= (travel-distance calais dover) 50)
46   (= (travel-distance calais newyork_port) 3500) (= (travel-distance
47     newyork_port calais) 3500)
48
49   (at-pkg p1 london)
50   (fragile p1)
51
52   (at-pkg p2 jfk)
53
54   (at-vehicle my_truck london)
55   (at-vehicle my_plane heathrow)
56   (at-vehicle my_ship dover)
57   (at-vehicle another_truck newyork_port)
58
59   (road-connection london heathrow) (road-connection heathrow london)
60   (road-connection london dover) (road-connection dover london)
61   (road-connection paris charlesdegaulle) (road-connection charlesdegaulle
    paris)
62   (road-connection paris calais) (road-connection calais paris)
63   (road-connection newyork jfk) (road-connection jfk newyork)
64   (road-connection newyork newyork_port) (road-connection newyork_port
    newyork)
65   (road-connection jfk newyork_port) (road-connection newyork_port jfk)
66   (road-connection london paris) (road-connection paris london)
67
68   (air-connection heathrow jfk) (air-connection jfk heathrow)
69   (air-connection heathrow charlesdegaulle) (air-connection charlesdegaulle
    heathrow)

```

```

62 (sea-connection dover calais) (sea-connection calais dover)
63 (sea-connection calais newyork_port) (sea-connection newyork_port calais)
64 )
65
66 (:goal
67   (and
68     (at-pkg p1 newyork)
69     (at-pkg p2 calais)
70   )
71 )
72
73 (:metric minimize (total-cost))
74 )

```

Listing 7: Definicja problemu zadania 1

2.3.1 Stan początkowy

W stanie początkowym zdefiniowano wartości dla fluentów numerycznych (koszt, czasy, prędkości, dystanse), położenie paczek i pojazdów, status paczki pierwszej jako delikatna oraz połączenia między lokacjami.

2.3.2 Cel i metryka

Celem było dostarczenie paczki z Londynu p1 do Nowego Jorku i paczki p2 z JFK do Calais, minimalizując całkowity koszt.

2.4 Wygenerowany plan

Testy przeprowadzono w edytorze online <https://editor.planning.domains> z użyciem solvera Temporal Fast Downward planning system. Poniżej znajduje się wygenerowany plan.

```

1 0.00100000: (drive-truck another_truck newyork_port newyork) [0.16666700]
2 0.00100000: (load-package p1 my_truck london) [1.00000000]
3 0.01100000: (drive-truck my_truck london heathrow) [0.50000000]
4 1.01100000: (unload-package p1 my_truck heathrow) [1.00000000]
5 0.17766700: (drive-truck another_truck newyork jfk) [0.41666700]
6 0.60433400: (drive-truck another_truck jfk newyork) [0.41666700]
7 2.02100000: (load-package p1 my_plane heathrow) [1.00000000]
8 1.03100100: (drive-truck another_truck newyork jfk) [0.41666700]
9 2.03100000: (fly-plane my_plane heathrow jfk) [6.00000000]
10 1.45766800: (drive-truck another_truck jfk newyork) [0.41666700]
11 1.88433500: (drive-truck another_truck newyork jfk) [0.41666700]
12 2.31100200: (drive-truck another_truck jfk newyork) [0.41666700]
13 2.73766900: (drive-truck another_truck newyork jfk) [0.41666700]
14 3.16433600: (drive-truck another_truck jfk newyork) [0.41666700]
15 3.59100300: (drive-truck another_truck newyork jfk) [0.41666700]
16 4.01767000: (drive-truck another_truck jfk newyork) [0.41666700]
17 4.44433700: (drive-truck another_truck newyork jfk) [0.41666700]
18 4.87100400: (drive-truck another_truck jfk newyork) [0.41666700]
19 5.29767100: (drive-truck another_truck newyork jfk) [0.41666700]
20 5.72433800: (drive-truck another_truck jfk newyork) [0.41666700]
21 6.15100500: (drive-truck another_truck newyork jfk) [0.41666700]
22 6.57767200: (drive-truck another_truck jfk newyork) [0.41666700]
23 8.04100000: (unload-package p1 my_plane jfk) [1.00000000]
24 7.00433900: (drive-truck another_truck newyork jfk) [0.41666700]
25 9.05200000: (load-package p1 another_truck jfk) [1.00000000]
26 9.06200000: (drive-truck another_truck jfk newyork) [0.41666700]
27 10.06200000: (unload-package p1 another_truck newyork) [1.00000000]
28 8.04200000: (load-package p2 my_plane jfk) [1.00000000]
29 1.02100000: (drive-truck my_truck heathrow london) [0.50000000]
30 8.05200000: (fly-plane my_plane jfk heathrow) [6.00000000]
31 0.00100000: (sail-ship my_ship dover calais) [1.66667000]
32 1.53100000: (drive-truck my_truck london dover) [1.33333000]
33 1.67767000: (sail-ship my_ship calais dover) [1.66667000]

```



```

34 2.87433000: (drive-truck my_truck dover london) [1.33333000]
35 4.21766000: (drive-truck my_truck london dover) [1.33333000]
36 3.35434000: (sail-ship my_ship dover calais) [1.66667000]
37 5.56099000: (drive-truck my_truck dover london) [1.33333000]
38 14.06200000: (unload-package p2 my_plane heathrow) [1.00000000]
39 6.90432000: (drive-truck my_truck london heathrow) [0.50000000]
40 15.07200000: (load-package p2 my_truck heathrow) [1.00000000]
41 15.08200000: (drive-truck my_truck heathrow london) [0.50000000]
42 15.59200000: (drive-truck my_truck london dover) [1.33333000]
43 5.03101000: (sail-ship my_ship calais dover) [1.66667000]
44 16.93533000: (unload-package p2 my_truck dover) [1.00000000]
45 17.94533000: (load-package p2 my_ship dover) [1.00000000]
46 17.95533000: (sail-ship my_ship dover calais) [1.66667000]
47 19.63200000: (unload-package p2 my_ship calais) [1.00000000]

```

Listing 8: Fragment wygenerowanego planu dla Zadania 1

2.5 Analiza planu i eksperymenty

- **Logika planu:** Planner poprawnie wybrał trasę multimodalną (ciężarówka do lotniska, samolot, ciężarówka z lotniska) w celu dostarczenia paczki.
- **Koszty i czas trwania:** Wprowadzenie kosztów pozwoliło na optymalizację planu pod kątem minimalizacji wydatków. Akcje czasowe z dynamicznie obliczanym czasem trwania (na podstawie dystansu i prędkości) umożliwiły realistyczne modelowanie operacji logistycznych i obliczenie całkowitego czasu realizacji (makespan).
- **Efekty warunkowe:** Dodatkowy koszt załadunku dla paczki oznaczonej jako delikatna ((fragile p1)) został poprawnie naliczony.
- **Zachowanie "niekrytycznych" agentów:** Zaobserwowano, że pojazd `another_truck`, który nie był na ścieżce krytycznej transportu paczki `p1` (ale był potrzebny na końcu), wykonywał w planie serię ruchów "jałowych". Jest to cecha planowania automatycznego, gdzie planner może wypełniać czas akcjami, które nie blokują osiągnięcia celu.

3 Zadanie 2: Sprzątający robot

3.1 Opis problemu

Zadanie polegało na stworzeniu modelu PDDL dla robota, który ma odwiedzić wszystkie zdefiniowane pokoje i posprzątać je. Robot może przemieszczać się między połączonymi pokojami i wykonywać akcję sprzątania w pokoju, w którym aktualnie się znajduje.

3.2 Definicja domeny (domain-task2.pddl)

```

1 (define (domain cleaning-robot-world)
2   (:requirements :strips :typing :negative-preconditions)
3
4   (:types
5     robot
6     room
7   )
8
9   (:predicates
10    (at ?r - robot ?p - room)
11    (is_dirty ?p - room)
12    (is_clean ?p - room)
13    (connected ?p1 - room ?p2 - room)
14  )
15

```

```

16 (:action move
17   :parameters (?r - robot ?from_room - room ?to_room - room)
18   :precondition (and
19     (at ?r ?from_room)
20     (connected ?from_room ?to_room)
21   )
22   :effect (and
23     (not (at ?r ?from_room))
24     (at ?r ?to_room)
25   )
26 )
27
28 (:action clean_current_room
29   :parameters (?r - robot ?p_to_clean - room)
30   :precondition (and
31     (at ?r ?p_to_clean)
32     (is_dirty ?p_to_clean)
33   )
34   :effect (and
35     (not (is_dirty ?p_to_clean))
36     (is_clean ?p_to_clean)
37   )
38 )
39 )

```

- Wymagania: :strips, :typing, :negative-preconditions.
- Typy: robot, room.
- Predykaty: (at ?r - robot ?p - room), (is_dirty ?p - room), (is_clean ?p - room), (connected ?p1 - room ?p2 - room).
- Akcje:
 - move (?r - robot ?from_room - room ?to_room - room): Przesuwa robota z ?from_room do ?to_room, jeśli są połączone i robot jest w ?from_room.
 - clean_current_room (?r - robot ?p_to_clean - room): Sprząta pokój ?p_to_clean, jeśli robot jest w tym pokoju i pokój jest brudny. Efektem jest zmiana statusu pokoju z is_dirty na is_clean.

3.3 Definicja problemu (problem-task2.pddl)

```

1 (define (problem clean-all-rooms-task)
2   (:domain cleaning-robot-world)
3
4   (:objects
5     r1 - robot
6     p1 p2 p3 - room
7   )
8
9   (:init
10    (at r1 p1)
11    (is_dirty p1)
12    (is_dirty p2)
13    (is_dirty p3)
14
15    (connected p1 p2) (connected p2 p1)
16    (connected p2 p3) (connected p3 p2)

```

```

17 )
18
19 (:goal
20   (and
21     (is_clean p1)
22     (is_clean p2)
23     (is_clean p3)
24   )
25 )
26 )

```

- **Obiekty:** r1 - robot, p1, p2, p3 - room.
- **Stan początkowy:** Robot w p1, wszystkie pokoje brudne. Zdefiniowano połączenia umożliwiające robotowi dotarcie do wszystkich pokoi (np. (connected p1 p2), (connected p2 p1), (connected p2 p3), (connected p3 p2)).
- **Cel:** Wszystkie pokoje są czyste: (and (is_clean p1) (is_clean p2) (is_clean p3)).

3.4 Wygenerowany plan

Do rozwiązania użyto solvera BFWS - FF-parser version. Wygenerowany plan:

```

1 (clean_current_room r1 p1)
2 (move r1 p1 p2)
3 (clean_current_room r1 p2)
4 (move r1 p2 p1)
5 (move r1 p1 p3)
6 (clean_current_room r1 p3)

```

Listing 9: Wygenerowany plan dla Zadania 2

Found Plan (output)

(clean_current_room r1 p1)
(move r1 p1 p2)
(clean_current_room r1 p2)
(move r1 p2 p1)
(move r1 p1 p3)
(clean_current_room r1 p3)

```

(:action clean_current_room
 :parameters (r1 p1)
 :precondition
   (and
     (at r1 p1)
     (is_dirty p1)
   )
 :effect
   (and
     (not
       (is_dirty p1)
     )
     (is_clean p1)
   )
 )

```

3.5 Analiza planu

Planner poprawnie wygenerował sekwencję akcji, która prowadzi do posprzątania wszystkich pokoi. Kluczowe było zdefiniowanie odpowiedniej topologii połączeń między pokojami, aby umożliwić robotowi dotarcie do każdego z nich. Jeśli początkowo połączenia były zdefiniowane tylko w jedną stronę lub nie tworzyły spójnego grafu, planner nie był w stanie znaleźć rozwiązania. Plan jest sekwencyjny i logiczny.

4 Zadanie 3: Robot z dwoma ramionami przenoszący piłki

4.1 Opis problemu

Zadanie polegało na analizie dostarczonego modelu PDDL dla robota z dwoma ramionami, który ma przenieść cztery piłki z jednego pokoju do drugiego. Robot może poruszać się między pokojami oraz podnosić i odkładać piłki każdym z ramion.

4.2 Analiza dostarczonych plików PDDL

Przeanalizowano dostarczone pliki `domain-task3.pddl` i `problem-task3.pddl`.

- **Domena:** Definiuje typy (`robot`, `room`, `ball`, `arm`) oraz predykaty opisujące stan świata (`at`, `inroom`, `holding`, `arm-empty`). Akcje to `move`, `pick-up` i `put-down`. Model jest klasycznym przykładem problemu STRIPS.
- **Problem:** Definiuje dwa pokoje, jednego robota (`robot1`), cztery piłki i dwa ramiona. Stan początkowy umieszcza robota i wszystkie piłki w `room1`, a ramiona są puste. Celem jest przeniesienie wszystkich piłek do `room2`.

W dostarczonym kodzie problemu poprawiono nazwę obiektu robota na `robot1` dla spójności.

4.3 Wygenerowany plan

Do rozwiązania użyto solvera LAMA-first: `satisficing planner`. Wygenerowany plan:

```
1 (pick-up robot1 arm1 ball1 room1)
2 (pick-up robot1 arm2 ball2 room1)
3 (move robot1 room1 room2)
4 (put-down robot1 arm1 ball1 room2)
5 (put-down robot1 arm2 ball2 room2)
6 (move robot1 room2 room1)
7 (pick-up robot1 arm1 ball3 room1)
8 (pick-up robot1 arm2 ball4 room1)
9 (move robot1 room1 room2)
10 (put-down robot1 arm1 ball3 room2)
11 (put-down robot1 arm2 ball4 room2)
```

Listing 10: Wygenerowany plan dla Zadania 3

Found Plan (output)

(pick-up robot1 arm1 ball1 room1)
(pick-up robot1 arm2 ball2 room1)
(move robot1 room1 room2)
(put-down robot1 arm1 ball1 room2)
(put-down robot1 arm2 ball2 room2)
(move robot1 room2 room1)
(pick-up robot1 arm1 ball3 room1)
(pick-up robot1 arm2 ball4 room1)
(move robot1 room1 room2)
(put-down robot1 arm1 ball3 room2)
(put-down robot1 arm2 ball4 room2)

```
(:action pick-up
:parameters (robot1 arm1 ball1 room1)
:precondition
  (and
    (at robot1 room1)
    (inroom ball1 room1)
    (arm-empty arm1)
  )
:effect
  (and
    (holding arm1 ball1)
    (not
      (arm-empty arm1)
    )
    (not
      (inroom ball1 room1)
    )
  )
)
```

4.4 Analiza planu

- **Wykorzystanie ramion:** Planner efektywnie wykorzystał oba ramiona robota, podnosząc dwie piłki przed każdym przejściem do drugiego pokoju. Skraca to liczbę koniecznych podróży między pokojami.
- **Optymalność:** Wygenerowany plan jest optymalny pod względem liczby kroków dla domyślnych ustawień planera STRIPS, który dąży do minimalizacji liczby akcji. Robot wykonuje minimalną liczbę podróży (dwie do pokoju docelowego i jedna z powrotem).
- **Logika:** Sekwencja akcji jest logiczna i bezpośrednio prowadzi do osiągnięcia celu. Robot najpierw zbiera maksymalną liczbę piłek, jaką może unieść (dwie), transportuje je, odkłada, a następnie wraca po kolejne. Niektóre solvery nie wykorzystywały tej optymalizacji.

5 Wnioski

Realizacja zadań z listy nr 3 pozwoliła na praktyczne zapoznanie się z językiem PDDL i jego zastosowaniami w automatycznym planowaniu.

- **Zalety PDDL:** Język jest wysoce ekspresywny, pozwala na deklaratywne opisanie problemu, oddzielając definicję świata od strategii jego rozwiązania. Rozszerzenia PDDL (czas, koszty, efekty warunkowe) umożliwiają modelowanie złożonych, realistycznych problemów.
- **Wady/Wyzwania PDDL:** Składnia, mimo że logiczna, wymaga precyzji i uwagi na detale. Debugowanie może być czasochłonne, a komunikaty błędów od planerów nie zawsze są jednoznaczne. Wybór odpowiedniego planera i jego konfiguracji jest kluczowy dla skutecznego rozwiązania problemu, zwłaszcza przy użyciu zaawansowanych rozszerzeń.
- **Napotkane trudności:** Główne trudności dotyczyły poprawnej składni zaawansowanych funkcji PDDL (np. efekty warunkowe w akcjach czasowych) i zapewnienia kompatybilności modelu z możliwościami wybranego solvera w edytorze online. Iteracyjne podejście do budowy modelu oraz analiza komunikatów błędów pozwoliły na przezwycięzenie tych problemów. Niestety, narzędzia do PDDL nie są w pełni dojrzałe - ich komunikaty błędów są mało wyraziste.

Praca z PDDL była cennym doświadczeniem, rozwijającym umiejętności modelowania logicznego i abstrakcyjnego myślenia o problemach. Wydaje się, że PDDL mógłby mieć mnóstwo zastosowań (agenci w grach, automatyka i robotyka, różne optymalizacje).